Graduation Project Document

January 2026

0.1 Currently Available Solutions (Applications)

0.1.1 What They Offer

Existing medical software can be categorized into three main groups based on their primary function:

Traditional Hospital Management Systems (e.g., Epic Systems, Cerner) These are the industry standards for managing large-scale hospital operations.

- **Offerings:** They excel in administrative tasks such as **Appointment Scheduling**, **Vital Signs Recording**, and **Allergy Checking**. They provide a robust digital version of a patient's paper chart, often called an Electronic Health Record (EHR).
- **Primary Benefit:** These systems are highly reliable for basic hospital logistics and regulatory compliance.

AI-Specialized Diagnostic Tools (e.g., Aidoc, PathAI) These platforms focus on specific medical imaging or laboratory tasks rather than general management.

- **Offerings:** They provide advanced analysis of visual data, such as X-rays or tissue samples, to help doctors spot abnormalities.
- **Primary Benefit:** They increase the speed and accuracy of specific diagnoses by flagging urgent cases.

General Health Data Platforms (e.g., IBM Watson Health, Google Health) These solutions focus on using large-scale data to provide insights.

- **Offerings:** They are strongest in **Clinical Consultation**, where the software can suggest potential treatments based on medical literature.
- **Primary Benefit:** They assist in high-level decision-making for complex cases.

Every Feature in a Separate Software!

0.1.2 Limitations and Disadvantages of Current Systems

The most significant challenge in the current medical software landscape is **fragmentation**. Currently, a healthcare provider cannot find a single all-in-one platform; instead, they must navigate a disjointed ecosystem of specialized tools. For example, if a doctor needs to schedule an appointment, they might use **Epic** or **Cerner**, but if they then require advanced AI analysis for a pathology report, they must switch to a different system like **PathAI**. This constant toggling between platforms creates technical friction, increases the likelihood of data entry errors, and wastes valuable clinical time.

But even if they decided to use many platforms, these solutions have limitations on their own! here are some of them:

- **Manual Data Bottlenecks:** Leading administrative systems like **Epic** and **Cerner** still require doctors to manually analyze patient histories and track progress over time, leading to administrative fatigue.
- **Absence of Proactive Monitoring:** Most established solutions lack effective **Case & Episode Tracking**, meaning they do not provide a continuous, automated view of a patient's recovery journey.

- **Limited Clinical Intelligence:** Platforms such as **IBM Watson Health** offer consultation features but fail to integrate essential administrative functions like **Allergy Checking** or **Multi-Role Support**.
- **Inadequate Prioritization:** Specialized AI tools like **Aidoc** excel at image analysis but cannot assist with **Risk-Based Prioritization** or **Evidence Generation**.
- **Communication Gaps:** A common weakness across platforms such as **Google Health** and **IBM** is the lack of **Real-Time Notifications**.

0.2 Features Matrix

Feature Comparison Matrix

Feature	**Epic Systems**	**Cerner**	**IBM Watson**	**Google Health**	**Aidoc**	**PathAI**	**Asclena (Ours)**
Patient Management	✓	✓	X	X	X	X	✓
Appointment Scheduling	✓	✓	X	X	X	X	✓
Case & Episode Tracking	Limited	Limited	X	X	X	X	✓
Vital Signs Recording	✓	✓	X	X	X	X	✓
Lab/Imaging Management	✓	✓	X	X	Limited	Limited	✓
AI Lab Analysis	X	X	Limited	X	X	✓	✓
AI Imaging Analysis	X	X	Limited	X	✓	X	✓
AI Clinical Consultation	X	X	✓	✓	X	X	✓
Patient History Analysis	Manual	Manual	Limited	X	X	X	✓ (AI)
Progress Comparison	Manual	Manual	X	X	X	X	✓ (AI)
Risk-Based Prioritization	X	X	X	X	X	X	✓ (AI)
Interactive Querying	X	X	Limited	Limited	X	X	✓
Evidence Generation	X	X	Limited	X	X	X	✓
Prescription Management	✓	✓	X	X	X	X	✓
Allergy Checking	✓	✓	X	X	X	X	✓
Multi-Role Support	✓	✓	X	X	Limited	Limited	✓
Audit Logging	✓	✓	Limited	X	Limited	Limited	✓
Real-Time Notifications	Limited	Limited	X	X	—	X	✓

Table 1: Comparison of features across major medical software platforms.

0.3 Software Development Methodology Used

0.3.1 Strategic Selection of Agile Kanban

For this project, we implemented **Agile Kanban** to manage the software development lifecycle (SDLC). This methodology was selected to prioritize a **continuous flow of delivery** over the rigid constraints of fixed-length iterations. By focusing on visualizing work and limiting Work-in-Progress (WIP), the team maximized throughput and maintained a steady delivery pace while remaining responsive to evolving requirements.
Selection Justification * **Process Efficiency**: Kanban was chosen for its ability to reduce process overhead by eliminating mandatory, time-intensive ceremonies, allowing the team to focus on high-priority development tasks. * **Visual Transparency**: The use of a visual workflow provided immediate clarity on task status, enabling the team to identify and resolve bottlenecks early in the development cycle.

0.3.2 Jira Implementation and Workflow Management

We utilized **Atlassian Jira** as our primary project management infrastructure to implement the Kanban framework. This platform served as the "single source of truth" for all development activities, providing several key advantages: **Workflow Visualization**: We configured a digital Kanban board with custom columns—**To Do, In Progress, In Review, and Done**—to map our specific development lifecycle. **Issue Tracking and Categorization**: Each work item (e.g., "CRUD Operations in NestJS" or "System Architecture") was tracked as a unique Jira issue, allowing for detailed documentation, sub-tasking, and ownership assignment.

1 Requirements Analysis

1.1 Surveys, Questionnaires, Interviews, etc.

1.1.1 Introduction

To ensure that the proposed medical AI assistant is aligned with real clinical needs and expectations, a structured survey was designed and distributed among medical professionals and medical trainees. Conducting this survey was a crucial step in the system development process, as it provided direct insight into doctors' perspectives on the use of Artificial Intelligence in clinical practice, their current workflows, their level of trust in AI-driven systems, and their expectations from such technologies.

AI-Powered Clinical Workflow and Prioritization Survey

A brief survey for medical professionals and students on the adoption of AI for patient history, case prioritization and task automation. Completion time: 5-7 minutes.

Administrative and Demographic Information The survey intentionally begins with basic demographic and professional questions (name, age, professional status, and contact information) to serve as a **soft and non-intrusive opening** for respondents. This approach helps clinicians ease into the survey before engaging with more detailed, cognitively demanding clinical and technical questions.

Collecting this information allows the research team to **contextualize responses** and understand *who* is providing the input, ensuring that insights are interpreted within the appropriate professional and experiential framework. Attributes such as age and professional status are particularly valuable for analyzing whether differences in clinical experience, career stage, or training background influence attitudes toward AI adoption, trust levels, and workflow expectations.

Full Name *

Short-answer text

Age (Numeric) *

Short-answer text

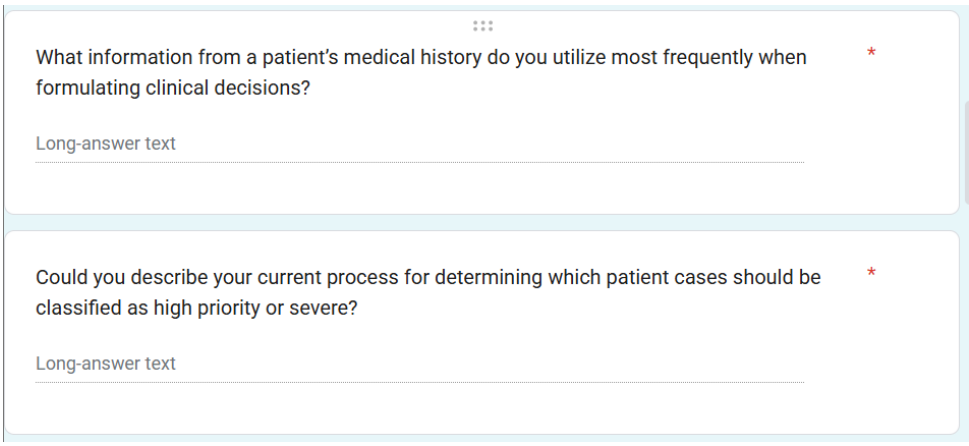
Phone Number

Short-answer text

The inclusion of a phone number is strictly optional and intended solely for **follow-up communication** in cases where clarification or additional insight is required. All personal data is handled confidentially, stored securely, and excluded from published analyses in accordance with data privacy and ethical research standards.

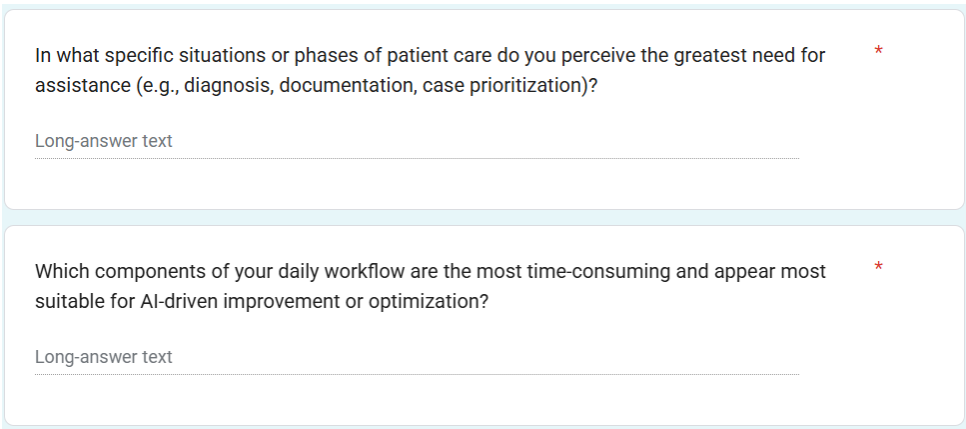
Clinical Decision-Making and Workflow Analysis Questions related to the respondent's professional role and the nature of their clinical work are essential for **understanding real-world workflows**. These insights directly inform system design decisions, including:

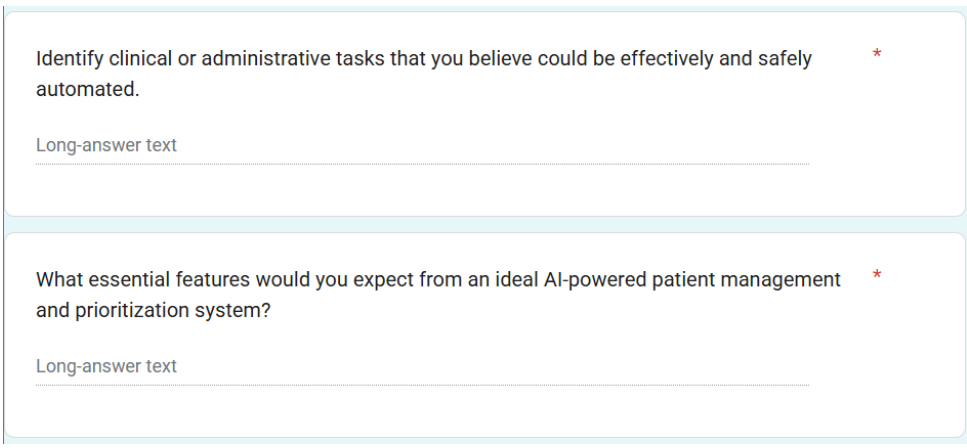
- How clinicians prioritize or classify cases as severe or high-risk
- Which elements of patient medical history are considered most clinically significant
- How decision-making processes vary across roles and healthcare settings



This information is also critical during the **AI model training and evaluation phases**, as it enables alignment between the system's outputs and actual clinical reasoning patterns. By grounding the model in practitioner-informed workflows, the system is better positioned to deliver accurate, relevant, and clinically actionable support.

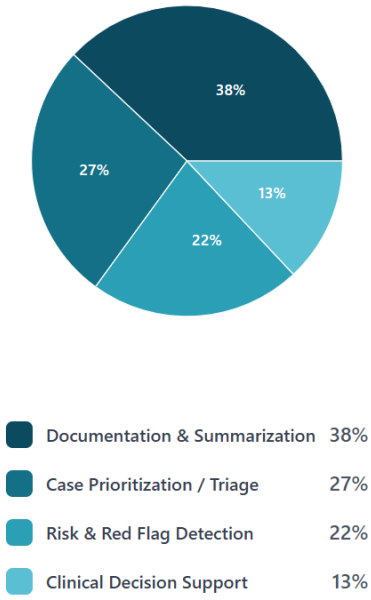
Needs Assessment and Feature Selection Strategy The survey also includes targeted questions that ask clinicians to identify the areas in which they most require assistance and where an AI-based solution could provide meaningful support. These questions are designed to capture real clinical pain points rather than assumed technical opportunities, ensuring that the proposed system addresses genuine needs within everyday medical practice.

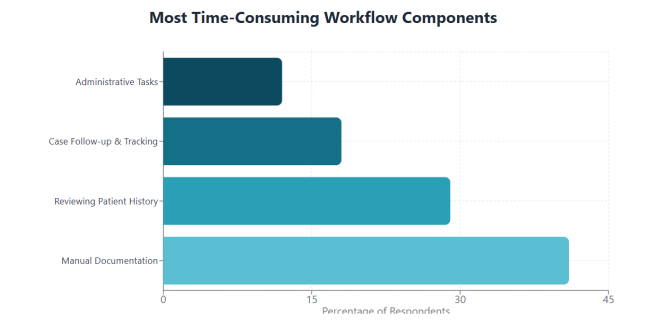


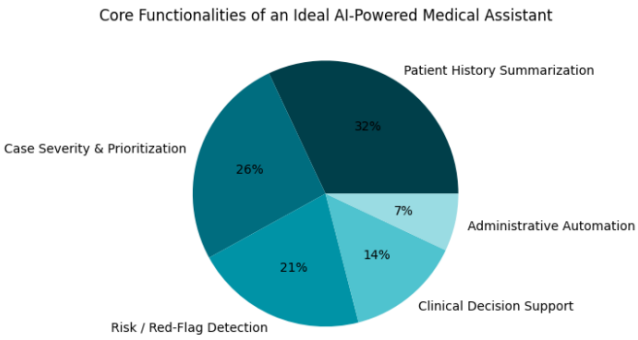


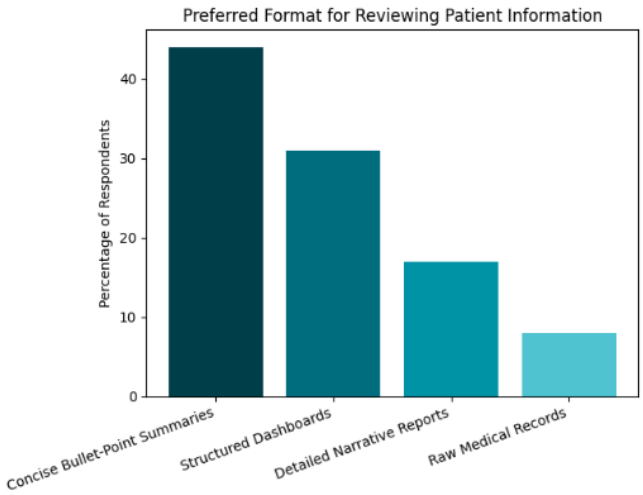
By conducting interviews before selecting system functionalities, the development process followed a **user-centered and evidence-driven approach**, allowing feature prioritization to be guided by clinician-identified needs rather than purely technical considerations. This methodology ensures that the resulting AI assistant focuses on the areas where support is most needed and where it can deliver the highest practical and clinical value.

1.2 Results Represented as Charts









1.3 Discussion of Results

Figure 1: Primary Areas Where Clinicians Need AI Assistance This diagram illustrates the areas in which clinicians reported the greatest need for AI-driven support. The highest demand is observed in **documentation and patient history summarization**, reflecting the significant cognitive and time burden associated with manual data synthesis. **Case prioritization and triage** also represent a major area of need, reinforcing the importance of AI-assisted severity assessment and workflow optimization.

Insight:

These findings directly justified prioritizing automated summarization and intelligent triage as core features of the proposed medical AI assistant.

Figure 2: Most Time-Consuming Workflow Components This visualization highlights the workflow components that clinicians identified as the most time-consuming. **Manual documentation** and **reviewing patient medical history** account for the majority of reported workload, followed by case tracking and administrative tasks.

Insight:

These findings strongly support the automation of documentation, history summarization, and patient tracking as high-impact, low-risk AI applications.

Figure 3: Core Functionalities of an Ideal AI-Powered Medical Assistant This diagram represents clinicians' vision of the most valuable capabilities in an ideal medical AI assistant. **Patient history summarization** emerged as the most desired feature, followed by **case severity assessment and prioritization** and **risk or red-flag detection**. Lower emphasis was placed on administrative automation, suggesting that clinicians prioritize clinical relevance over purely operational features.

Interpretation:

These findings validate the decision to center the system around **clinical intelligence and decision support**, rather than treating it solely as an administrative automation tool.

Figure 4: Preferred Format for Reviewing Patient Information This chart illustrates clinicians' preferred formats for reviewing patient data. The majority of respondents favored **concise bullet-point**

summaries, followed by **structured dashboards**, indicating a strong preference for rapid information consumption and reduced cognitive load. Detailed narrative reports were less preferred, while raw medical records were selected by only a small percentage of respondents.

Interpretation:

These results reinforce the importance of delivering AI outputs in a **clear, summarized, and structured format**, allowing clinicians to quickly extract actionable insights without navigating excessive detail.

1.4 1. Functional Requirements

System Behavior and User Features

1.4.1 1.1 Patient History Management

- **Document Upload**: The system shall allow Doctors to upload patient medical history documents for processing.
- **Automated AI Analysis**: The system shall use an AI model to examine uploaded documents and generate a comprehensive overview of the patient's condition.
- **Progress Tracking**: The system shall compare current medical reports with historical data to determine if a condition is improving, worsening, or stable.
- **Interactive Querying**: Doctors shall be able to submit natural language prompts to query specific details within a patient's historical records.

1.4.2 1.2 Clinical Decision Support

- **Treatment Recommendations**: Based on analyzed data, the system shall provide potential treatment recommendations to assist medical professionals.
- **Risk-Based Prioritization**: The system shall rank a doctor's assigned patients based on the severity of their condition to ensure urgent cases are handled first.
- **Review & Validation**: Doctors must be able to review and validate AI-generated insights before any clinical action is finalized.

1.4.3 1.3 Administrative & Security Operations

- **Role-Based Access Control (RBAC)**: The system shall support distinct user roles, including Doctors, Technicians, and Administrators, managed via a dedicated Auth & User Module.
- **Audit Logging**: The system shall maintain historical records of all uploaded documents, AI recommendations, and doctor validations for longitudinal tracking.

1.5 2. Non-Functional Requirements

System Constraints, Quality Attributes, and Technical Standards

1.5.1 2.1 Security and Privacy

Access Control Enforcement

The system shall enforce role-based access control at the request handling layer to ensure that only authorized users (Doctor, Technician, and Administrator) can access their respective functional modules.

Input Validation and Sanitization

All incoming requests shall undergo systematic validation and data normalization to ensure correctness, consistency, and protection against malformed or malicious inputs before being processed by core business services.

Regulatory and Ethical Compliance

The system shall comply with applicable data protection regulations and medical ethics standards to guarantee the confidentiality, integrity, and lawful handling of sensitive patient health information.

1.5.2 2.2 Performance and Reliability

Asynchronous Task Handling

Time-intensive operations, including AI-driven analysis and document processing, shall be executed asynchronously as background tasks to maintain responsiveness of the primary application services.

Caching and Task Coordination

An in-memory caching and queuing mechanism shall be employed to improve data access performance, reduce redundant processing, and efficiently coordinate background workloads.

Data Consistency and Persistence

A relational database management system shall be used to ensure reliable data storage, transactional consistency, and structural integrity for complex and interrelated medical records.

1.5.3 2.3 Scalability and Maintainability

Modular System Design

The system shall adopt a modular architecture, organizing functionality into well-defined domains (e.g., Authentication, Patient Management, Document Handling, and AI Assistance) to improve maintainability and support future architectural evolution.

Event-Driven Workflow Execution

Internal system interactions shall leverage event-based communication to trigger background processes, enabling loose coupling between components and improved scalability.

Centralized Error Handling

A unified error management mechanism shall be implemented to consistently capture, process, and report runtime exceptions, ensuring system stability and meaningful operational feedback.

1.5.4 2.4 Operational Key Performance Indicators (KPIs)

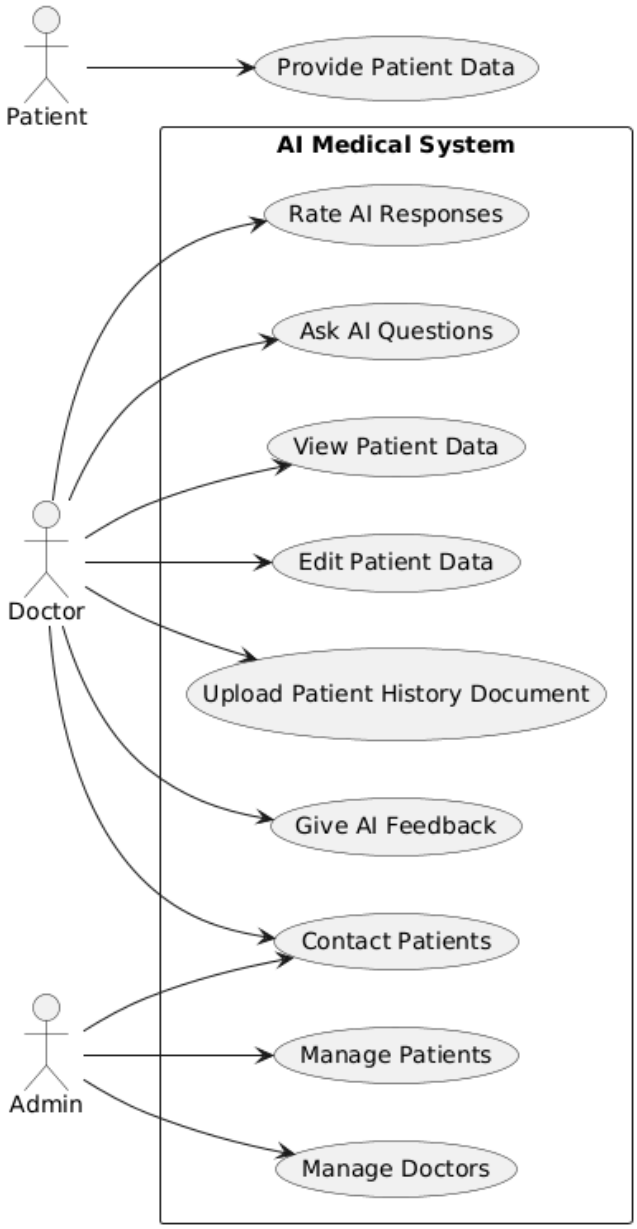
Clinical Decision Support Efficiency

The system shall aim to reduce the average time required for medical professionals to review patient histories and reach informed clinical decisions.

System Availability and Data Reliability

The system shall maintain high availability for document processing and AI-assisted analysis while ensuring minimal risk of data loss during storage, retrieval, and processing operations

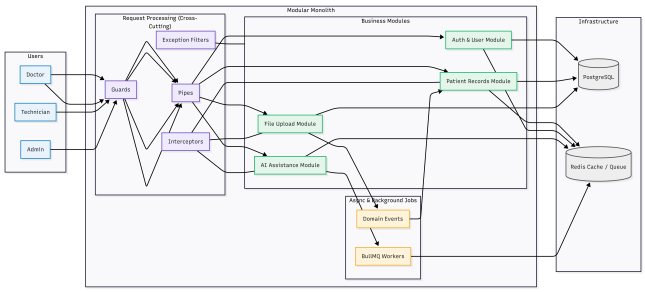
1.6 Use Case Diagram



2 Implementation Aspects

2.1 Overall System Architecture (Blocks diagram)

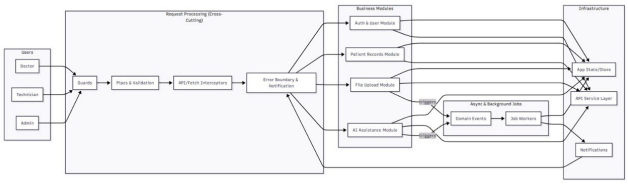
2.1.1 Asclena Backend System Architecture



The architecture transitions away from a traditional "spaghetti" monolith toward a **Modular Monolith**. This pattern balances the simplicity of a single deployment unit with the maintainability of microservices by enforcing

strict logical boundaries between business domains. The inclusion of asynchronous processing and cross-cutting request logic indicates a high level of maturity regarding performance and security.

2.1.2 Asclena Frontend System Architecture



NextJS: The architecture follows a modern web application pattern with clear separation of concerns across presentation, business logic, and infrastructure layers. The system serves three distinct user types (Doctor, Technician, and Admin) through a unified interface while maintaining appropriate access controls and data segregation.

2.2 Tools, Technologies and/or Programming Languages that will be used illustrated on the diagram in (Part 4.a)

2.2.1 Asclena Backend System tools

A. Request Processing (The "NestJS" Pipeline) The system utilizes a robust cross-cutting layer to handle incoming requests from Doctors, Technicians, and Admins:

- **Guards:** Primary defense for authentication and Role-Based Access Control (RBAC).
- **Pipes:** Ensures data integrity through validation and transformation before reaching business logic.
- **Interceptors/Exception Filters:** These handle global concerns like response mapping, logging, and standardized error handling, ensuring the business modules remain "clean."

B. Business Modules (Domain Logic) The core logic is partitioned into four distinct modules:

- **Auth & User Module:** Centralizes identity management.
- **Patient Records Module:** The primary domain, likely handling CRUD operations and sensitive medical history.
- **File Upload Module:** Segregated to handle multi-part data or large assets (e.g., medical scans).
- **AI Assistance Module:** An isolated bridge for intelligent features, preventing heavy AI processing logic from blocking standard record-keeping functions.

C. Async & Background Jobs This is a critical strength of the architecture. By utilizing **Domain Events** and **BullMQ Workers**, the system can:

- Offload time-consuming tasks (like AI processing or file optimization).
- Improve user-perceived latency by returning success responses before the background work is finished.
- Ensure reliability through Redis-backed job retries.

D. Infrastructure Layer The dual-database approach is optimal for this use case:

- **PostgreSQL:** Provides ACID compliance and relational integrity for patient data.
- **Redis:** Serves as both a high-speed cache for session data and the message broker for the job queue.

2.2.2 Asclena Frontend System tools

2.2.3 1. Presentation Layer

Request Processing Pipeline

The application implements a cross-cutting request processing layer that intercepts all incoming requests before they reach business logic. This layer consists of three sequential stages:

- Guards: Implements authentication and authorization checks, ensuring users have appropriate credentials and permissions before processing requests
- Pipes & Validation: Performs input sanitization and validation, transforming raw request data into typed, validated objects
- API/Fetch Interceptors: Handles request/response transformation, error normalization, and potentially adds headers or telemetry data

Error Boundary & Notification System

A centralized error handling mechanism captures exceptions from any layer and routes them appropriately. Errors trigger notifications through a dedicated notification service, providing user feedback while maintaining system stability.

2.2.4 2. Business Modules Layer

The application separates business concerns into discrete, cohesive modules:

Authentication & User Module Manages user identity, session management, and role-based access control. This module authenticates all three user types and maintains their session state throughout the application lifecycle.

Patient Records Module Handles the core healthcare data management functionality, including patient demographics, medical histories, treatment records, and clinical documentation. This is likely the most data-intensive module given the healthcare context.

File Upload Module Manages document and media uploads associated with patient records, such as medical images, lab results, consent forms, and other clinical documentation. Implements file validation, storage management, and retrieval mechanisms.

AI Assistance Module Provides intelligent features leveraging machine learning or AI capabilities, potentially including clinical decision support, diagnostic assistance, predictive analytics, or natural language processing for medical documentation.

2.2.5 3. Infrastructure Layer

Async & Background Jobs Processing The system implements an event-driven architecture for handling long-running or deferred operations:

- Domain Events: Application-specific events that represent significant business occurrences (e.g., "Patient Record Updated", "File Upload Completed")
- Job Workers: Background processors that consume events and execute tasks asynchronously, preventing UI blocking and improving system responsiveness

This pattern is particularly important for resource-intensive operations like AI model inference, large file processing, or batch data operations.

API Service Layer Serves as the communication bridge between the frontend application and backend services. This layer abstracts HTTP communication details and provides a consistent interface for data operations across all business modules.

App State/Store Implements centralized state management for the application. This ensures consistent data representation across components and manages the application's reactive data flow.

Notifications A dedicated service for user-facing notifications, handling success messages, error alerts, and informational updates. Integrates with the error boundary system to provide contextual feedback.

2.3 Data Flow Architecture

The architecture demonstrates clear unidirectional data flow:

1. User actions originate from role-specific interfaces (Doctor, Technician, Admin)
2. Requests pass through the validation pipeline (Guards → Pipes → Interceptors)
3. Business modules process the validated requests
4. Infrastructure services handle data persistence, state updates, and async operations
5. Responses flow back through the API layer to update the application state
6. UI components reactively update based on state changes

2.4 Technology Stack Implications

As a Next.js application, this architecture benefits from:

- Server-side rendering capabilities for improved initial load performance
- API routes for backend logic without separate server infrastructure
- Automatic code splitting for optimal bundle sizes
- Built-in optimization for production deployments

2.5 Conclusion

This Next.js architecture demonstrates mature software design principles with appropriate separation of concerns, security considerations, and scalability provisions. The modular structure supports maintainability while the async processing capabilities ensure responsive user experience. The healthcare domain focus requires continued attention to data privacy, security, and regulatory compliance as the system evolves.